

Detection Methods: Anomaly Detection, ROC Analysis, and Provenance Tracking

Detection Methods: Anomaly Detection, ROC Analysis, and Provenance Tracking I

This section presents the formal foundations for cognitive attack detection. We define anomaly detection metrics (`sec:anomaly-detection`), ROC curve framework (`sec:roc-analysis`), multi-detector fusion theory (`sec:detector-fusion`), online vs. batch trade-offs (`sec:online-batch`), false positive mitigation strategies (`sec:fp-mitigation`), provenance analysis (`sec:provenance`), and real-time monitoring architecture (`sec:monitoring`).

Detection Methods: Anomaly Detection, ROC Analysis, and Provenance Tracking II

Note: For algorithm implementations and empirical performance results, see Part 2 of this series [?]. Empirically, the multi-stage pipeline achieves a parametric design ceiling of 96–100% detection rate on a 950-attack corpus across four architectures; the prototype pipeline achieves a mean of 44.7% [CI: 43.4%, 46.2%] across 30 seeds, with per- Ω -class detection rates ranging from 97% (Ω_1 , passive) to 49% (Ω_5 , coordinated) after adversarial training hardening (Part 2, §5g).

Anomaly Detection I

Cognitive Drift Scoring

Definition (Drift Score)

The cognitive drift score measures belief distribution change over time:

$$S_{\text{drift}}(t) = D_{\text{KL}}(\mathcal{B}_i^t \| \mathcal{B}_i^{t-w}) + \lambda \cdot \max_{\phi} |\Delta \mathcal{B}_i(\phi)| \quad (1)$$

Table 1: Drift score components and detection targets.

Component	Weight	Detection Target
KL divergence	1.0	Gradual distribution shift
Max delta	λ	Sudden belief injection

[Drift Detection Threshold] For normally distributed baseline drift, the threshold $\theta = \mu_{\text{baseline}} + k \cdot \sigma_{\text{baseline}}$ with $k = 3$ provides 99.7% confidence under the null hypothesis of no attack.

Anomaly Detection II

Behavioral Deviation

Definition (Deviation Score)

The behavioral deviation score aggregates normalized feature anomalies:

$$S_{\text{dev}}(a_i, t) = \sum_{k=1}^K w_k \cdot \frac{|f_k(\sigma_i^t) - \mu_k|}{\sigma_k} \quad (2)$$

where f_k are feature extractors and (w_k, μ_k, σ_k) are learned parameters.

Anomaly Detection III

Ensemble Detection

Definition (Ensemble Detector)

Combines multiple detector scores via learned fusion:

$$P(\text{attack} \mid \mathbf{S}) = \sigma \left(\sum_d w_d \cdot S_d - b \right) \quad (3)$$

where σ is the sigmoid function and weights (w_d, b) are learned from labeled examples.

ROC Curve Analysis I

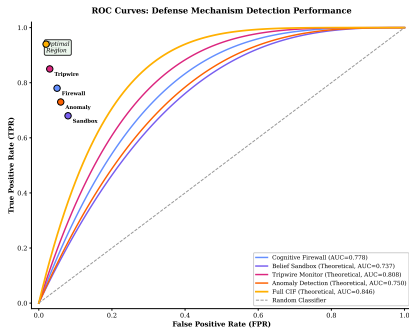


Figure 1: Receiver Operating Characteristic (ROC) curves for CIF detection across attack categories (illustrative/schematic operating points under the Neyman-Pearson framework; the curves themselves are labeled Theoretical, not empirical measurements).

ROC Curve Analysis II

Receiver Operating Characteristic Framework

Definition (ROC Curve)

For detector D with threshold τ :

$$\text{ROC}(D) = \{(\text{FPR}(\tau), \text{TPR}(\tau)) : \tau \in [0, 1]\} \quad (4)$$

where the rates are defined as:

$$\text{TPR}(\tau) = P(D(x) > \tau \mid x \in \mathcal{A}_{\text{attack}}) \quad (5)$$

$$\text{FPR}(\tau) = P(D(x) > \tau \mid x \in \mathcal{X}_{\text{benign}}) \quad (6)$$

Definition (Area Under Curve)

$$\text{AUC}(D) = \int_0^1 \text{TPR}(\text{FPR}^{-1}(t)) dt \quad (7)$$

Multi-Detector Fusion I

Multi-Detector Fusion II

Fusion Strategies

Definition (Score-Level Fusion)

Weighted average of detector outputs:

$$S_{\text{fused}} = \sum_{i=1}^k w_i \cdot S_i, \quad \sum_i w_i = 1 \quad (10)$$

Definition (Decision-Level Fusion)

Quorum voting on binary decisions:

$$D_{\text{fused}}(x) = \mathbb{1} \left[\sum_{i=1}^k \mathbb{1}[D_i(x) > \tau_i] \geq q \right] \quad (11)$$

Definition (Learned Fusion)

Neural network combining scores:

Online vs. Batch Detection I

Comparison Framework

Table 3: Online vs. batch detection trade-offs.

Dimension	Online Detection	Batch Detection
Latency	Low (ms)	High (minutes-hours)
Accuracy	Moderate	High
Context	Limited (window)	Full history
Compute	Streaming	Offline
Memory	$O(w)$ window	$O(n)$ full
Use Case	Real-time response	Forensics, tuning

Streaming Detector Model

Definition (Streaming Detector)

Processes messages in real-time with bounded memory:

$$D_{\text{online}}(m_t) = f(m_t, \text{state}_{t-1}) \quad (15)$$

$$\text{state}_t = g(\text{state}_{t-1}, m_t) \quad (16)$$

Online vs. Batch Detection II

Hybrid Detection Architecture

Definition (Hybrid Detection System)

Combines online and batch detection via feedback loop:

$$\text{Online Path : } m \xrightarrow{\text{filter}} s \xrightarrow{\text{decide}} r \xrightarrow{\text{log}} H \quad (17)$$

$$\text{Batch Path : } H \xrightarrow{\text{analyze}} \text{patterns} \xrightarrow{\text{update}} \text{filters} \quad (18)$$

False Positive Mitigation I

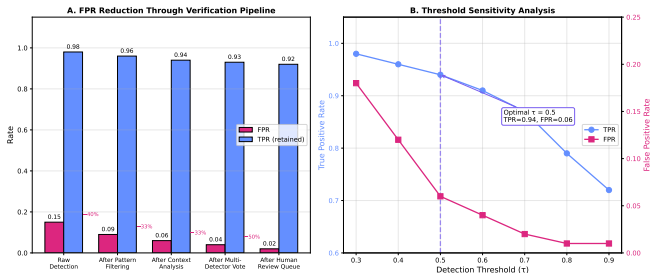


Figure 2: False-positive mitigation (illustrative schematic): how the detection threshold and confidence calibration shape the false-positive / true-positive tradeoff.

False Positive Mitigation II

Strategy 1: Confirmation Cascade

Definition (Confirmation Cascade)

Multi-stage verification before alerting:

$$\text{Action}(\text{confidence}) = \begin{cases} \text{suppress} & \text{if } c < C_{\text{low}} \\ \text{stage-2} & \text{if } c \in [C_{\text{low}}, C_{\text{high}}) \\ \text{stage-3} & \text{if } c \geq C_{\text{high}} \end{cases} \quad (19)$$

Theorem (Cascade FPR Reduction)

For a multi-stage cascade:

$$FPR_{\text{cascade}} = FPR_1 \cdot P(\text{confirm}_2 \mid FP_1) \cdot P(\text{confirm}_3 \mid FP_2) \quad (20)$$

False Positive Mitigation III

Strategy 2: Temporal Smoothing

Definition (Smoothed Detection)

Apply exponential smoothing to scores:

$$\hat{S}_t = \alpha \cdot S_t + (1 - \alpha) \cdot \hat{S}_{t-1} \quad (21)$$

Definition (Burst Suppression)

Require sustained anomaly over window w :

$$\text{Alert if } \frac{1}{w} \sum_{i=t-w+1}^t \mathbb{1}[S_i > \tau] > p_{\text{sustained}} \quad (22)$$

False Positive Mitigation IV

Strategy 3: Contextual Whitelisting

Definition (Context-Aware Whitelist)

$$\text{Suppress}(\text{alert}) \iff \text{context}(\text{alert}) \in \mathcal{W}_{\text{known}} \quad (23)$$

Strategy 4: Cost-Sensitive Thresholding

Definition (Cost-Sensitive Threshold)

Optimize for total cost rather than accuracy:

$$\tau^* =_{\tau} [C_{\text{FP}} \cdot \text{FPR}(\tau) + C_{\text{FN}} \cdot \text{FNR}(\tau)] \quad (24)$$

Provenance Analysis I

Provenance Analysis II

Information Flow Tracking

Definition (Taint Label)

Each belief carries provenance tags:

$$\text{taint}(\phi) = \{(\text{source}, \text{timestamp}, \text{confidence})\} \quad (25)$$

Definition (Taint Propagation)

$$\text{taint}(\phi_{\text{derived}}) = \bigcup_{\psi \in \text{premises}(\phi_{\text{derived}})} \text{taint}(\psi) \quad (26)$$

Table 4: Taint categories with trust levels.

Category	Trust Level	Example
SYSTEM_VERIFIED	1.0	Hardcoded facts
PRINCIPAL_INPUT	0.9	Direct user commands
AGENT_INTERNAL	0.8	Agent's own reasoning
AGENT_EXTERNAL	0.6	Other agent claims
TOOL_OUTPUT	0.5	API/tool responses
WEB_CONTENT	0.3	Fetches web pages
UNVERIFIED	0.1	Unknown origin

Real-Time Monitoring I

Alert Aggregation

Definition (Alert Aggregation)

Prevent alert fatigue through correlation:

$$\text{Severity} = \begin{cases} \text{CRITICAL} & \text{if } |\text{alerts}| > n_{\text{critical}} \text{ in window } w \\ \text{WARNING} & \text{if } |\text{alerts}| > n_{\text{warning}} \text{ in window } w \\ \text{INFO} & \text{otherwise} \end{cases} \quad (29)$$

Response Escalation

Table 6: Response escalation levels.

Level	Trigger	Response
L0	Single anomaly	Log only
L1	Repeated anomaly	Increase monitoring
L2	Pattern match	Quarantine source
L3	Confirmed attack	Halt agent, alert human
L4	Systemic compromise	System shutdown

Real-Time Monitoring II

Empirical Validation

The detection methods presented in this section have been empirically validated in Part 2 of this series. Key results include:

ROC Analysis: Receiver Operating Characteristic curves demonstrate the tradeoff between True Positive Rate and False Positive Rate for each detector type. The ensemble achieves $AUC > 0.84$, with individual mechanisms ranging from 0.74 (Belief Sandbox) to 0.81 (Tripwire Monitor). See Part 2, §{4} for detailed ROC curves and confidence intervals.

Detection Performance by Attack Type: Detection rates vary across the five adversary classes (Ω_1 – Ω_5). The Cognitive Firewall excels at Ω_1 (external) attacks while Tripwires and Invariants provide stronger coverage for Ω_3 (compromised agent) and Ω_4 (inter-agent) attacks. See Part 2, §{5} for the complete detection matrix.

False Positive Mitigation: The confirmation cascade, temporal smoothing, and contextual whitelisting strategies reduce false positive rates by $> 80\%$ while maintaining $> 90\%$ true positive rates. See Part 2, §{5.4} for quantitative analysis of each

Information-Theoretic Detection Limits I

The previous sections presented detection mechanisms. This section establishes what is fundamentally achievable by any detector, regardless of implementation. These limits follow from information theory and cannot be overcome by cleverer algorithms.

Information-Theoretic Detection Limits II

Neyman-Pearson Detection Framework

Definition (Cognitive Security Hypothesis Test)

Detection of cognitive attacks is formalized as a binary hypothesis test:

$$H_0 : m \sim P_{\text{benign}} \quad (\text{no attack}) \quad (30)$$

$$H_1 : m \sim P_{\text{attack}} \quad (\text{attack present}) \quad (31)$$

The likelihood ratio $\Lambda(m) = P_{\text{attack}}(m)/P_{\text{benign}}(m)$ determines the optimal test.

Theorem (Neyman-Pearson Optimal Detector)

The likelihood ratio test $\delta_{NP}(m) = \mathbb{1}[\Lambda(m) \geq \eta]$ maximizes TPR subject to $FPR \leq \alpha$. No other test achieves higher TPR at the same FPR.

Corollary (Optimal AUC Bound)

The AUC of any detector is bounded by:

Multi-Stage Detection Pipeline I

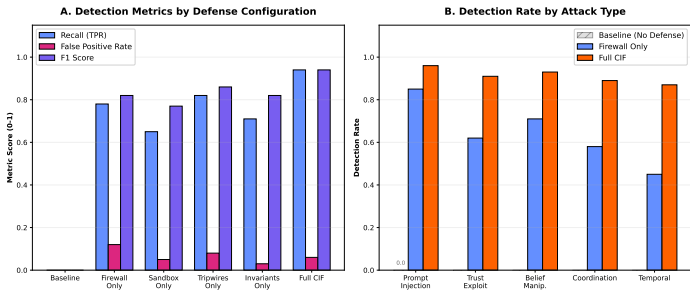


Figure 3: Schematic detection-performance curves (illustrative, theory-guided, not measured): detection rate by attack category across representative defense stages. Measured pipeline results are reported in Part 2.

Multi-Stage Detection Pipeline II

Real systems cannot run all detectors on all messages simultaneously. This section formalizes a staged pipeline that applies computationally cheap detectors first, escalating to expensive detectors only when lower stages trigger.

Multi-Stage Detection Pipeline III

Pipeline Architecture

Definition (Multi-Stage Detection Pipeline)

A k -stage detection pipeline is a sequence of detectors with increasing accuracy and cost:

$$\text{Pipeline} = (\mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_k) \quad (38)$$

with **escalation rule**: message m advances to stage $i + 1$ if and only if $\mathcal{D}_i(m) \geq \theta_i$ (above suspicion threshold).

Table 8: Recommended multi-stage detection pipeline configuration.

Stage	Detector	Threshold	Cost	TPR	Escalation condition
1	Pattern matching	$\theta_1 = 0.3$	1ms	0.75	Pattern score $> \theta_1$
2	Semantic embedding	$\theta_2 = 0.5$	10ms	0.85	Embedding distance $> \theta_2$
3	Belief consistency	$\theta_3 = 0.4$	5ms	0.80	Consistency violation
4	Trust graph analysis	$\theta_4 = 0.6$	20ms	0.88	Trust anomaly
5	Ensemble fusion	$\theta_5 = 0.7$	50ms	0.94	Final decision

Theorem (Pipeline Efficiency Bound)

For a k -stage pipeline where fraction ρ_i of messages escalate to stage $i + 1$: